

Chapter 3: Production-Ready Backend - Real-time, Testing & Deployment

Excellent progress! Now let's build a **production-grade backend** with real-time features, caching, testing, and deployment strategies.

Chapter 3 Learning Goals

By the end of this chapter, you'll master:

- **Real-time Communication** (WebSockets, Socket.io)
- **Caching Strategies** (Redis, Memory caching)
- **API Documentation** (Swagger/OpenAPI)
- **Testing** (Unit, Integration, E2E tests)
- **Monitoring & Logging** (Advanced logging, health checks)
- **Performance Optimization**
- **Production Deployment** (Docker, CI/CD, Environment setup)
- **Security Hardening**

Real-time Communication with Socket.IO

1. Setup WebSocket Server

```
npm install socket.io cors
```

[Copy](#)

```
// server.js - Updated with Socket.IO
require('dotenv').config();

const app = require('./src/app');
const connectDB = require('./src/config/database');
const logger = require('./src/utils/logger');
const { createServer } = require('http');
const { Server } = require('socket.io');

const PORT = process.env.PORT || 3000;

// Create HTTP server
const server = createServer(app);

// Setup Socket.IO
const io = new Server(server, {
```

[Copy](#)

```
cors: {
  origin: process.env.CLIENT_URL || "http://localhost:3000",
  methods: ["GET", "POST"],
  credentials: true
},
pingTimeout: 60000,
pingInterval: 25000
});

// Connect to database
connectDB();

// Socket.IO connection handling
io.on('connection', (socket) => {
  logger.info(`User connected: ${socket.id}`);

  // Join user to their personal room
  socket.on('join', (userId) => {
    socket.join(`user_${userId}`);
    socket.userId = userId;
    logger.info(`User ${userId} joined personal room`);
  });

  // Join chat room
  socket.on('join_room', (roomId) => {
    socket.join(`room_${roomId}`);
    socket.to(`room_${roomId}`).emit('user_joined', {
      userId: socket.userId,
      message: 'User joined the room'
    });
  });

  // Handle new message
  socket.on('send_message', async (data) => {
    try {
      const message = {
        id: Date.now(),
        userId: socket.userId,
        roomId: data.roomId,
        text: data.text,
        timestamp: new Date()
      };

      // Save message to database
      await saveMessageToDB(message);

      // Emit to all users in room
      io.to(`room_${data.roomId}`).emit('new_message', message);
    } catch (error) {
      logger.error('Error saving message to database:', error);
    }
  });
});
```

```
    } catch (error) {
      socket.emit('error', { message: 'Failed to send message' });
    }
  });

// Handle typing indicator
socket.on('typing_start', (data) => {
  socket.to(`room_${data.roomId}`).emit('user_typing', {
    userId: socket.userId,
    isTyping: true
  });
});

socket.on('typing_stop', (data) => {
  socket.to(`room_${data.roomId}`).emit('user_typing', {
    userId: socket.userId,
    isTyping: false
  });
});

// Handle disconnect
socket.on('disconnect', () => {
  logger.info(`User disconnected: ${socket.id}`);
});

// Helper function to save message
async function saveMessageToDB(message) {
  // Implement based on your Message model
  const Message = require('./src/models/Message.model');
  return await Message.create(message);
}

// Start server
server.listen(PORT, () => {
  logger.info(`Server running on port ${PORT} in ${process.env.NODE_ENV} mode`);
});

// Export io for use in controllers
app.set('io', io);
```

2. Real-time Notifications System

```
// models/Notification.model.js
const mongoose = require('mongoose');

const notificationSchema = new mongoose.Schema({
```

[Copy](#)

```
    recipient: {
      type: mongoose.Schema.ObjectId,
      ref: 'User',
      required: true
    },
    sender: {
      type: mongoose.Schema.ObjectId,
      ref: 'User',
      required: true
    },
    type: {
      type: String,
      enum: ['like', 'comment', 'follow', 'post', 'system'],
      required: true
    },
    message: {
      type: String,
      required: true
    },
    relatedPost: {
      type: mongoose.Schema.ObjectId,
      ref: 'Post'
    },
    isRead: {
      type: Boolean,
      default: false
    },
    data: {
      type: mongoose.Schema.Types.Mixed,
      default: {}
    }
  }, {
    timestamps: true
  });

// Index for efficient queries
notificationSchema.index({ recipient: 1, createdAt: -1 });
notificationSchema.index({ recipient: 1, isRead: 1 });

module.exports = mongoose.model('Notification', notificationSchema);
```

```
// services/notification.service.js
const Notification = require('../models/Notification.model');

class NotificationService {

  // Create and send notification
```

 Copy

```
static async createAndSend(io, notificationData) {
  try {
    const notification = await Notification.create(notificationData);
    await notification.populate('sender', 'name avatar');

    // Send real-time notification
    io.to(`user_${notificationData.recipient}`).emit('new_notification', {
      id: notification._id,
      type: notification.type,
      message: notification.message,
      sender: notification.sender,
      createdAt: notification.createdAt,
      data: notification.data
    });

    return notification;
  } catch (error) {
    console.error('Failed to create notification:', error);
  }
}

// Send like notification
static async sendLikeNotification(io, postAuthorId, likerId, postId) {
  if (postAuthorId.toString() === likerId.toString()) return;

  return this.createAndSend(io, {
    recipient: postAuthorId,
    sender: likerId,
    type: 'like',
    message: 'Liked your post',
    relatedPost: postId
  });
}

// Send comment notification
static async sendCommentNotification(io, postAuthorId, commenterId, postId,
comment) {
  if (postAuthorId.toString() === commenterId.toString()) return;

  return this.createAndSend(io, {
    recipient: postAuthorId,
    sender: commenterId,
    type: 'comment',
    message: 'commented on your post',
    relatedPost: postId,
    data: { comment: comment.substring(0, 50) + '...' }
  });
}
```

```
// Send follow notification
static async sendFollowNotification(io, followedUserId, followerId) {
  return this.createAndSend(io, {
    recipient: followedUserId,
    sender: followerId,
    type: 'follow',
    message: 'started following you'
  });
}
}

module.exports = NotificationService;
```

3. Real-time Integration in Controllers

```
// Update post.controller.js with real-time features
const NotificationService = require('../services/notification.service');

const likePost = async (req, res) => {
  try {
    const post = await Post.findById(req.params.id);

    if (!post) {
      return res.status(404).json({
        success: false,
        message: 'Post not found'
      });
    }

    const alreadyLiked = post.likes.find(like =>
      like.user.toString() === req.user.id
    );

    if (alreadyLiked) {
      post.likes = post.likes.filter(like =>
        like.user.toString() !== req.user.id
      );
    } else {
      post.likes.push({ user: req.user.id });

      // Send real-time notification
      const io = req.app.get('io');
      await NotificationService.sendLikeNotification(
        io,
        post.author._id,
        req.user.id,
        post._id
      );
    }
  }
}
```

 Copy

```
    );  
  }  
  
  await post.save();  
  
  // Emit real-time update to all clients following this post  
  const io = req.app.get('io');  
  io.emit('post_updated', {  
    postId: post._id,  
    likesCount: post.likes.length,  
    action: alreadyLiked ? 'unlike' : 'like',  
    userId: req.user.id  
  });  
  
  res.status(200).json({  
    success: true,  
    data: post,  
    message: alreadyLiked ? 'Post unliked' : 'Post liked'  
  });  
} catch (error) {  
  res.status(500).json({  
    success: false,  
    message: error.message  
  });  
}  
};
```

Caching with Redis

1. Redis Setup & Configuration

```
npm install redis ioredis
```

 Copy

```
// config/redis.js  
const Redis = require('ioredis');  
const logger = require('../utils/logger');  
  
class RedisClient {  
  constructor() {  
    this.client = null;  
    this.isConnected = false;  
  }  
}
```

 Copy

```
async connect() {
  try {
    this.client = new Redis({
      host: process.env.REDIS_HOST || 'localhost',
      port: process.env.REDIS_PORT || 6379,
      password: process.env.REDIS_PASSWORD,
      db: process.env.REDIS_DB || 0,
      retryDelayOnFailover: 100,
      enableOfflineQueue: false,
      maxRetriesPerRequest: 3,
      lazyConnect: true
    });

    this.client.on('connect', () => {
      this.isConnected = true;
      logger.info('Connected to Redis successfully');
    });

    this.client.on('error', (error) => {
      this.isConnected = false;
      logger.error('Redis connection error:', error);
    });

    this.client.on('close', () => {
      this.isConnected = false;
      logger.warn('Redis connection closed');
    });

    await this.client.connect();
    return this.client;
  } catch (error) {
    logger.error('Failed to connect to Redis:', error);
    throw error;
  }
}

async disconnect() {
  if (this.client) {
    await this.client.quit();
    this.isConnected = false;
  }
}

getClient() {
  return this.client;
}

isHealthy() {
  return this.isConnected && this.client.status === 'ready';
}
```

```
}  
}  
  
module.exports = new RedisClient();
```

2. Cache Service Layer

```
// services/cache.service.js  
const redisClient = require('../config/redis');  
const logger = require('../utils/logger');  
  
class CacheService {  
  
  // Get data from cache  
  async get(key) {  
    try {  
      if (!redisClient.isHealthy()) return null;  
  
      const data = await redisClient.getClient().get(key);  
      return data ? JSON.parse(data) : null;  
    } catch (error) {  
      logger.error(`Cache GET error for key ${key}:`, error);  
      return null;  
    }  
  }  
  
  // Set data in cache  
  async set(key, data, ttl = 3600) {  
    try {  
      if (!redisClient.isHealthy()) return false;  
  
      const serializedData = JSON.stringify(data);  
  
      if (ttl > 0) {  
        await redisClient.getClient().setex(key, ttl, serializedData);  
      } else {  
        await redisClient.getClient().set(key, serializedData);  
      }  
  
      return true;  
    } catch (error) {  
      logger.error(`Cache SET error for key ${key}:`, error);  
      return false;  
    }  
  }  
  
  // Delete from cache
```

 Copy

```
async del(key) {
  try {
    if (!redisClient.isHealthy()) return false;

    await redisClient.getClient().del(key);
    return true;
  } catch (error) {
    logger.error(`Cache DELETE error for key ${key}:`, error);
    return false;
  }
}

// Delete multiple keys with pattern
async delPattern(pattern) {
  try {
    if (!redisClient.isHealthy()) return false;

    const keys = await redisClient.getClient().keys(pattern);
    if (keys.length > 0) {
      await redisClient.getClient().del(...keys);
    }

    return true;
  } catch (error) {
    logger.error(`Cache DELETE pattern error for ${pattern}:`, error);
    return false;
  }
}

// Increment counter
async incr(key, ttl = 3600) {
  try {
    if (!redisClient.isHealthy()) return null;

    const value = await redisClient.getClient().incr(key);
    if (value === 1 && ttl > 0) {
      await redisClient.getClient().expire(key, ttl);
    }

    return value;
  } catch (error) {
    logger.error(`Cache INCR error for key ${key}:`, error);
    return null;
  }
}

// Get cache statistics
async getStats() {
  try {
```

```

if (!redisClient.isHealthy()) return null;

const info = await redisClient.getClient().info();
const keyspace = await redisClient.getClient().info('keyspace');

return {
  connected: true,
  info: info.split('\n').reduce((acc, line) => {
    const [key, value] = line.split(':');
    if (key && value) acc[key] = value.trim();
    return acc;
  }, {}),
  keyspace
};
} catch (error) {
  logger.error('Failed to get Redis stats:', error);
  return { connected: false, error: error.message };
}
}
}

module.exports = new CacheService();

```

3. Caching Middleware

```

// middleware/cache.middleware.js
const cacheService = require('../services/cache.service');
const logger = require('../utils/logger');

// Generic cache middleware
const cache = (options = {}) => {
  const {
    ttl = 3600,
    keyGenerator = (req) => `${req.method}:${req.originalUrl}`,
    condition = () => true
  } = options;

  return async (req, res, next) => {
    try {
      // Skip caching if condition not met
      if (!condition(req)) {
        return next();
      }

      const cacheKey = keyGenerator(req);

      // Try to get from cache

```

 Copy

```
const cachedData = await cacheService.get(cacheKey);

if (cachedData) {
  logger.info(`Cache HIT for key: ${cacheKey}`);
  return res.status(200).json({
    ...cachedData,
    cached: true,
    cacheKey
  });
}

logger.info(`Cache MISS for key: ${cacheKey}`);

// Override res.json to cache the response
const originalJson = res.json;
res.json = function(data) {
  // Only cache successful responses
  if (res.statusCode === 200 && data.success) {
    cacheService.set(cacheKey, data, ttl);
  }

  return originalJson.call(this, data);
};

next();
} catch (error) {
  logger.error('Cache middleware error:', error);
  next();
}
};

// Specific cache middleware for different use cases
const cacheUserProfile = cache({
  ttl: 1800, // 30 minutes
  keyGenerator: (req) => `user:profile:${req.user.id}`,
  condition: (req) => req.method === 'GET'
});

const cachePostList = cache({
  ttl: 600, // 10 minutes
  keyGenerator: (req) => {
    const page = req.query.page || 1;
    const limit = req.query.limit || 10;
    const search = req.query.search || '';
    return `posts:list:${page}:${limit}:${search}`;
  },
  condition: (req) => req.method === 'GET' && !req.query.nocache
});
```

```
const cacheStats = cache({
  ttl: 3600, // 1 hour
  keyGenerator: () => 'stats:general',
  condition: (req) => req.method === 'GET'
});

module.exports = {
  cache,
  cacheUserProfile,
  cachePostList,
  cacheStats
};
```

4. Cache Invalidation Strategy

```
// services/cache-invalidation.service.js
const cacheService = require('./cache.service');
const logger = require('../utils/logger');

class CacheInvalidationService {

  // Invalidate user-related cache
  static async invalidateUserCache(userId) {
    const patterns = [
      `user:profile:${userId}`,
      `user:posts:${userId}*`,
      'posts:list:*', // Invalidate all post lists
      'stats:*' // Invalidate stats
    ];

    for (const pattern of patterns) {
      await cacheService.delPattern(pattern);
    }

    logger.info(`Invalidated cache for user: ${userId}`);
  }

  // Invalidate post-related cache
  static async invalidatePostCache(postId, authorId) {
    const patterns = [
      `post:${postId}`,
      `user:posts:${authorId}*`,
      'posts:list:*',
      'stats:*'
    ];
  }
```

 Copy

```
for (const pattern of patterns) {
  await cacheService.delPattern(pattern);
}

logger.info(`Invalidated cache for post: ${postId}`);
}

// Invalidate all stats cache
static async invalidateStatsCache() {
  await cacheService.delPattern('stats:*');
  logger.info('Invalidated all stats cache');
}
}

module.exports = CacheInvalidationService;
```

API Documentation with Swagger

1. Setup Swagger

```
npm install swagger-jsdoc swagger-ui-express
```

[Copy](#)

```
// config/swagger.js
const swaggerJsdoc = require('swagger-jsdoc');
const swaggerUi = require('swagger-ui-express');

const options = {
  definition: {
    openapi: '3.0.0',
    info: {
      title: 'MyApp API Documentation',
      version: '1.0.0',
      description: 'A comprehensive API for social media platform',
      contact: {
        name: 'API Support',
        email: 'support@myapp.com',
        url: 'https://myapp.com/support'
      }
    },
  },
  servers: [
    {
      url: process.env.API_URL || 'http://localhost:3000',
      description: 'Development server'
    }
  ]
};
```

[Copy](#)

```
    },
    {
      url: 'https://api.myapp.com',
      description: 'Production server'
    }
  ],
  components: {
    securitySchemes: {
      bearerAuth: {
        type: 'http',
        scheme: 'bearer',
        bearerFormat: 'JWT'
      }
    },
    schemas: {
      User: {
        type: 'object',
        properties: {
          id: {
            type: 'string',
            description: 'Unique identifier'
          },
          name: {
            type: 'string',
            description: 'User full name'
          },
          email: {
            type: 'string',
            format: 'email',
            description: 'User email address'
          },
          role: {
            type: 'string',
            enum: ['user', 'admin'],
            description: 'User role'
          },
          createdAt: {
            type: 'string',
            format: 'date-time',
            description: 'Account creation date'
          }
        }
      },
      Post: {
        type: 'object',
        properties: {
          id: {
            type: 'string',
            description: 'Unique identifier'
          }
        }
      }
    }
  }
}
```

```

    },
    title: {
      type: 'string',
      description: 'Post title'
    },
    content: {
      type: 'string',
      description: 'Post content'
    },
    author: {
      $ref: '#/components/schemas/User'
    },
    likesCount: {
      type: 'integer',
      description: 'Number of likes'
    },
    createdAt: {
      type: 'string',
      format: 'date-time'
    }
  },
},
Error: {
  type: 'object',
  properties: {
    success: {
      type: 'boolean',
      example: false
    },
    message: {
      type: 'string',
      description: 'Error message'
    },
    errors: {
      type: 'array',
      items: {
        type: 'object',
        properties: {
          field: { type: 'string' },
          message: { type: 'string' }
        }
      }
    }
  }
}
},
},
},
},
},
},
},
apis: ['./src/routes/*.js', './src/models/*.js'] // Paths to files with OpenAPI

```

```
definitions
};

const specs = swaggerJsdoc(options);

module.exports = {
  specs,
  swaggerUi
};
```

2. Documented Routes Example

```
// routes/auth.routes.js with Swagger documentation
const express = require('express');
const { register, login, getMe } = require('../controllers/auth.controller');
const { protect } = require('../middleware/auth.middleware');
const { validate } = require('../middleware/validation.middleware');
const { registerSchema, loginSchema } = require('../validators/auth.validator');

const router = express.Router();
```

 Copy

```
/**
 * @swagger
 * /api/v1/auth/register:
 *   post:
 *     summary: Register a new user
 *     tags: [Authentication]
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             required:
 *               - name
 *               - email
 *               - password
 *             properties:
 *               name:
 *                 type: string
 *                 example: John Doe
 *               email:
 *                 type: string
 *                 format: email
 *                 example: john@example.com
 *               password:
 *                 type: string
```

```

*           minLength: 6
*           example: password123
*   responses:
*     201:
*       description: User registered successfully
*       content:
*         application/json:
*           schema:
*             type: object
*             properties:
*               success:
*                 type: boolean
*                 example: true
*               token:
*                 type: string
*                 description: JWT authentication token
*               data:
*                 type: object
*                 properties:
*                   user:
*                     $ref: '#/components/schemas/User'
*     400:
*       description: Validation error
*       content:
*         application/json:
*           schema:
*             $ref: '#/components/schemas/Error'
*/
router.post('/register', validate(registerSchema), register);

/**
 * @swagger
 * /api/v1/auth/login:
 *   post:
 *     summary: Login user
 *     tags: [Authentication]
 *     requestBody:
 *       required: true
 *     content:
 *       application/json:
 *         schema:
 *           type: object
 *           required:
 *             - email
 *             - password
 *           properties:
 *             email:
 *               type: string
 *               format: email

```

```

*           example: john@example.com
*           password:
*           type: string
*           example: password123
* responses:
*   200:
*     description: Login successful
*     content:
*       application/json:
*         schema:
*           type: object
*           properties:
*             success:
*               type: boolean
*               example: true
*             token:
*               type: string
*             data:
*               type: object
*               properties:
*                 user:
*                   $ref: '#/components/schemas/User'
*   401:
*     description: Invalid credentials
*     content:
*       application/json:
*         schema:
*           $ref: '#/components/schemas/Error'
*/
router.post('/login', validate(loginSchema), login);

/**
 * @swagger
 * /api/v1/auth/me:
 *   get:
 *     summary: Get current user profile
 *     tags: [Authentication]
 *     security:
 *       - bearerAuth: []
 *     responses:
 *       200:
 *         description: User profile retrieved successfully
 *         content:
 *           application/json:
 *             schema:
 *               type: object
 *               properties:
 *                 success:
 *                   type: boolean

```

```

*           example: true
*           data:
*             $ref: '#/components/schemas/User'
*
* 401:
*     description: Unauthorized
*     content:
*       application/json:
*         schema:
*           $ref: '#/components/schemas/Error'
*/
router.get('/me', protect, getMe);

module.exports = router;

```

3. Add Swagger to App

```

// Update app.js to include Swagger
const { specs, swaggerUi } = require('./config/swagger');

// Add after other middleware
app.use('/api/docs', swaggerUi.serve, swaggerUi.setup(specs, {
  explorer: true,
  customCss: '.swagger-ui .topbar { display: none }',
  customSiteTitle: 'MyApp API Documentation'
}));

// Redirect /docs to /api/docs
app.get('/docs', (req, res) => {
  res.redirect('/api/docs');
});

```

 Copy

Comprehensive Testing Suite

1. Testing Setup

```

npm install --save-dev jest supertest @babel/preset-env
npm install --save-dev mongodb-memory-server

```

 Copy

```

// jest.config.js
module.exports = {
  testEnvironment: 'node',

```

 Copy

```

setupFilesAfterEnv: ['<rootDir>/tests/setup.js'],
testMatch: [
  '<rootDir>/tests/**/*.test.js'
],
collectCoverageFrom: [
  'src/**/*.js',
  '!src/config/**',
  '!**/node_modules/**'
],
coverageDirectory: 'coverage',
coverageReporters: ['text', 'lcov', 'html'],
verbose: true,
detectOpenHandles: true,
forceExit: true,
clearMocks: true,
resetMocks: true,
restoreMocks: true
};

```

```

// tests/setup.js
const { MongoMemoryServer } = require('mongodb-memory-server');
const mongoose = require('mongoose');

let mongod;

// Setup before all tests
beforeAll(async () => {
  // Start in-memory MongoDB
  mongod = await MongoMemoryServer.create();
  const uri = mongod.getUri();

  // Connect mongoose to in-memory db
  await mongoose.connect(uri, {
    useNewUrlParser: true,
    useUnifiedTopology: true
  });
});

// Cleanup after each test
afterEach(async () => {
  const collections = mongoose.connection.collections;

  for (const key in collections) {
    await collections[key].deleteMany({});
  }
});

```

 Copy

```
// Cleanup after all tests
afterAll(async () => {
  await mongoose.connection.dropDatabase();
  await mongoose.connection.close();
  await mongod.stop();
});

// Global test timeout
jest.setTimeout(30000);
```

2. Unit Tests

```
// tests/unit/services/auth.service.test.js
const User = require('../../src/models/User.model');
const authService = require('../../src/services/auth.service');

describe('Auth Service', () => {
  describe('createUser', () => {
    it('should create user with hashed password', async () => {
      const userData = {
        name: 'John Doe',
        email: 'john@example.com',
        password: 'password123'
      };

      const user = await authService.createUser(userData);

      expect(user).toBeDefined();
      expect(user.email).toBe(userData.email);
      expect(user.password).not.toBe(userData.password); // Should be hashed
      expect(user.password.length).toBeGreaterThan(10);
    });

    it('should throw error for duplicate email', async () => {
      const userData = {
        name: 'John Doe',
        email: 'john@example.com',
        password: 'password123'
      };

      await authService.createUser(userData);

      await expect(authService.createUser(userData))
        .rejects
        .toThrow('Email already exists');
    });
  });
});
```

 Copy

```

describe('validatePassword', () => {
  let user;

  beforeEach(async () => {
    user = await User.create({
      name: 'John Doe',
      email: 'john@example.com',
      password: 'password123'
    });
  });

  it('should return true for correct password', async () => {
    const isValid = await authService.validatePassword(user, 'password123');
    expect(isValid).toBe(true);
  });

  it('should return false for incorrect password', async () => {
    const isValid = await authService.validatePassword(user, 'wrongpassword');
    expect(isValid).toBe(false);
  });
});

describe('generateJWT', () => {
  it('should generate valid JWT token', () => {
    const payload = { id: '507f1f77bcf86cd799439011' };
    const token = authService.generateJWT(payload);

    expect(token).toBeDefined();
    expect(typeof token).toBe('string');
    expect(token.split('.').length).toBe(3); // JWT has 3 parts
  });
});

```

3. Integration Tests

```

// tests/integration/auth.test.js
const request = require('supertest');
const app = require('../../src/app');
const User = require('../../src/models/User.model');

describe('Authentication Endpoints', () => {
  describe('POST /api/v1/auth/register', () => {
    it('should register a new user successfully', async () => {
      const userData = {
        name: 'John Doe',

```

 Copy

```
    email: 'john@example.com',
    password: 'password123'
  };

  const response = await request(app)
    .post('/api/v1/auth/register')
    .send(userData)
    .expect(201);

  expect(response.body.success).toBe(true);
  expect(response.body.token).toBeDefined();
  expect(response.body.data.user.email).toBe(userData.email);
  expect(response.body.data.user.name).toBe(userData.name);

  // Verify user was created in database
  const user = await User.findOne({ email: userData.email });
  expect(user).toBeDefined();
});

it('should return validation errors for invalid data', async () => {
  const invalidData = {
    name: 'Jo', // Too short
    email: 'invalid-email', // Invalid format
    password: '123' // Too short
  };

  const response = await request(app)
    .post('/api/v1/auth/register')
    .send(invalidData)
    .expect(400);

  expect(response.body.success).toBe(false);
  expect(response.body.errors).toBeDefined();
  expect(response.body.errors.length).toBeGreaterThan(0);
});

it('should reject duplicate email registration', async () => {
  const userData = {
    name: 'John Doe',
    email: 'john@example.com',
    password: 'password123'
  };

  // First registration
  await request(app)
    .post('/api/v1/auth/register')
    .send(userData)
    .expect(201);
```

```
// Second registration with same email
const response = await request(app)
  .post('/api/v1/auth/register')
  .send(userData)
  .expect(400);

expect(response.body.success).toBe(false);
expect(response.body.message).toContain('email');
});
});

describe('POST /api/v1/auth/login', () => {
  let user;

  beforeEach(async () => {
    user = await User.create({
      name: 'John Doe',
      email: 'john@example.com',
      password: 'password123'
    });
  });

  it('should login with valid credentials', async () => {
    const loginData = {
      email: 'john@example.com',
      password: 'password123'
    };

    const response = await request(app)
      .post('/api/v1/auth/login')
      .send(loginData)
      .expect(200);

    expect(response.body.success).toBe(true);
    expect(response.body.token).toBeDefined();
    expect(response.body.data.user.email).toBe(loginData.email);
  });

  it('should reject invalid credentials', async () => {
    const loginData = {
      email: 'john@example.com',
      password: 'wrongpassword'
    };

    const response = await request(app)
      .post('/api/v1/auth/login')
      .send(loginData)
      .expect(401);
```

```
    expect(response.body.success).toBe(false);
    expect(response.body.message).toContain('Invalid credentials');
  });
});

describe('GET /api/v1/auth/me', () => {
  let user;
  let token;

  beforeEach(async () => {
    user = await User.create({
      name: 'John Doe',
      email: 'john@example.com',
      password: 'password123'
    });

    token = user.getSignedJwtToken();
  });

  it('should get user profile with valid token', async () => {
    const response = await request(app)
      .get('/api/v1/auth/me')
      .set('Authorization', `Bearer ${token}`)
      .expect(200);

    expect(response.body.success).toBe(true);
    expect(response.body.data.email).toBe(user.email);
    expect(response.body.data.name).toBe(user.name);
  });

  it('should reject request without token', async () => {
    const response = await request(app)
      .get('/api/v1/auth/me')
      .expect(401);

    expect(response.body.success).toBe(false);
    expect(response.body.message).toContain('Not authorized');
  });

  it('should reject request with invalid token', async () => {
    const response = await request(app)
      .get('/api/v1/auth/me')
      .set('Authorization', 'Bearer invalid-token')
      .expect(401);

    expect(response.body.success).toBe(false);
    expect(response.body.message).toContain('Not authorized');
  });
});
```

```
});
```

4. Test Helper Utilities

```
// tests/helpers/auth.helper.js
const User = require('../../src/models/User.model');
const jwt = require('jsonwebtoken');

class AuthHelper {
  // Create test user
  static async createTestUser(userData = {}) {
    const defaultUser = {
      name: 'Test User',
      email: 'test@example.com',
      password: 'password123',
      role: 'user'
    };

    return await User.create({ ...defaultUser, ...userData });
  }

  // Create admin user
  static async createTestAdmin(userData = {}) {
    return await this.createTestUser({
      ...userData,
      role: 'admin',
      email: userData.email || 'admin@example.com'
    });
  }

  // Generate JWT token for user
  static generateTestToken(userId) {
    return jwt.sign({ id: userId }, process.env.JWT_SECRET, {
      expiresIn: process.env.JWT_EXPIRE || '7d'
    });
  }

  // Get authorization header
  static getAuthHeader(token) {
    return { Authorization: `Bearer ${token}` };
  }
}

module.exports = AuthHelper;
```

 Copy



Monitoring & Health Checks

1. Health Check System

```
// routes/health.routes.js
const express = require('express');
const mongoose = require('mongoose');
const redisClient = require('../config/redis');
const cacheService = require('../services/cache.service');

const router = express.Router();

// Basic health check
router.get('/', (req, res) => {
  res.status(200).json({
    status: 'OK',
    timestamp: new Date().toISOString(),
    uptime: process.uptime(),
    environment: process.env.NODE_ENV,
    version: process.env.npm_package_version || '1.0.0'
  });
});

// Detailed health check
router.get('/detailed', async (req, res) => {
  const health = {
    status: 'OK',
    timestamp: new Date().toISOString(),
    uptime: process.uptime(),
    environment: process.env.NODE_ENV,
    version: process.env.npm_package_version || '1.0.0',
    services: {}
  };

  // Check MongoDB connection
  try {
    const dbState = mongoose.connection.readyState;
    health.services.mongodb = {
      status: dbState === 1 ? 'UP' : 'DOWN',
      state: ['disconnected', 'connected', 'connecting', 'disconnecting'][dbState]
    };
  } catch (error) {
    health.services.mongodb = {
      status: 'DOWN',
      error: error.message
    };
  }
});
```

[Copy](#)

```
// Check Redis connection
try {
  if (redisClient.isHealthy()) {
    const redisStats = await cacheService.getStats();
    health.services.redis = {
      status: 'UP',
      ...redisStats
    };
  } else {
    health.services.redis = {
      status: 'DOWN',
      error: 'Redis not connected'
    };
  }
} catch (error) {
  health.services.redis = {
    status: 'DOWN',
    error: error.message
  };
}

// Check memory usage
const memUsage = process.memoryUsage();
health.system = {
  memory: {
    rss: Math.round(memUsage.rss / 1024 / 1024) + ' MB',
    heapTotal: Math.round(memUsage.heapTotal / 1024 / 1024) + ' MB',
    heapUsed: Math.round(memUsage.heapUsed / 1024 / 1024) + ' MB',
    external: Math.round(memUsage.external / 1024 / 1024) + ' MB'
  },
  cpu: process.cpuUsage()
};

// Determine overall health
const allServicesUp = Object.values(health.services)
  .every(service => service.status === 'UP');

health.status = allServicesUp ? 'OK' : 'DEGRADED';

const statusCode = allServicesUp ? 200 : 503;
res.status(statusCode).json(health);
});

// Ready check (for Kubernetes readiness probe)
router.get('/ready', async (req, res) => {
  try {
    // Check if essential services are available
    const dbReady = mongoose.connection.readyState === 1;
```

```
if (dbReady) {
  res.status(200).json({ status: 'Ready' });
} else {
  res.status(503).json({ status: 'Not Ready', reason: 'Database not connected'
});
}
} catch (error) {
  res.status(503).json({ status: 'Not Ready', error: error.message });
}
});

// Live check (for Kubernetes liveness probe)
router.get('/live', (req, res) => {
  res.status(200).json({
    status: 'Alive',
    timestamp: new Date().toISOString()
  });
});

module.exports = router;
```

Production Deployment

1. Docker Configuration

```
# Dockerfile
# Build stage
FROM node:18-alpine AS builder

WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm ci --only=production && npm cache clean --force

# Production stage
FROM node:18-alpine AS production

# Create app directory
WORKDIR /app

# Create non-root user
```

 Copy

```
RUN addgroup -g 1001 -S nodejs && \
    adduser -S nodejs -u 1001

# Copy package files and node_modules from builder
COPY package*.json ./
COPY --from=builder /app/node_modules ./node_modules

# Copy application code
COPY --chown=nodejs:nodejs . .

# Create necessary directories
RUN mkdir -p logs uploads && \
    chown -R nodejs:nodejs /app

# Switch to non-root user
USER nodejs

# Expose port
EXPOSE 3000

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
    CMD node healthcheck.js

# Start application
CMD ["node", "server.js"]
```

```
# docker-compose.yml
version: '3.8'

services:
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - MONGODB_URI=mongodb://mongo:27017/myapp
      - REDIS_HOST=redis
      - REDIS_PORT=6379
    depends_on:
      - mongo
      - redis
    restart: unless-stopped
    networks:
      - app-network
```

 Copy

```
mongo:
  image: mongo:5.0
  ports:
    - "27017:27017"
  volumes:
    - mongo_data:/data/db
  environment:
    - MONGO_INITDB_ROOT_USERNAME=admin
    - MONGO_INITDB_ROOT_PASSWORD=password
  restart: unless-stopped
  networks:
    - app-network
```

```
redis:
  image: redis:7-alpine
  ports:
    - "6379:6379"
  volumes:
    - redis_data:/data
  restart: unless-stopped
  networks:
    - app-network
```

```
nginx:
  image: nginx:alpine
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf
    - ./ssl:/etc/nginx/ssl
  depends_on:
    - app
  restart: unless-stopped
  networks:
    - app-network
```

```
volumes:
  mongo_data:
  redis_data:
```

```
networks:
  app-network:
    driver: bridge
```

2. Environment Configuration



```
// config/environment.js
const environments = {
  development: {
    port: 3000,
    database: {
      uri: process.env.MONGODB_URI || 'mongodb://localhost:27017/myapp-dev',
      options: {
        useNewUrlParser: true,
        useUnifiedTopology: true
      }
    },
  },
  redis: {
    host: process.env.REDIS_HOST || 'localhost',
    port: process.env.REDIS_PORT || 6379
  },
  jwt: {
    secret: process.env.JWT_SECRET || 'dev-secret',
    expiresIn: '1d'
  },
  cors: {
    origin: ['http://localhost:3000', 'http://localhost:3001']
  },
  logging: {
    level: 'debug'
  }
},

production: {
  port: process.env.PORT || 8080,
  database: {
    uri: process.env.MONGODB_URI,
    options: {
      useNewUrlParser: true,
      useUnifiedTopology: true,
      maxPoolSize: 10,
      serverSelectionTimeoutMS: 5000,
      socketTimeoutMS: 45000,
      bufferMaxEntries: 0,
      bufferCommands: false
    }
  },
  redis: {
    host: process.env.REDIS_HOST,
    port: process.env.REDIS_PORT || 6379,
    password: process.env.REDIS_PASSWORD
  },
  jwt: {
    secret: process.env.JWT_SECRET,
```

```
    expiresIn: process.env.JWT_EXPIRE || '7d'
  },
  cors: {
    origin: process.env.ALLOWED_ORIGINS?.split(',') || []
  },
  logging: {
    level: 'info'
  },
  ssl: {
    enabled: process.env.SSL_ENABLED === 'true',
    cert: process.env.SSL_CERT_PATH,
    key: process.env.SSL_KEY_PATH
  }
}
};

const env = process.env.NODE_ENV || 'development';

module.exports = environments[env];
```

Chapter 3 Summary

You've now mastered **production-ready backend development**:

Real-time Features

- WebSocket integration with Socket.IO
- Real-time notifications system
- Live chat and messaging
- Real-time data updates

Performance Optimization

- Redis caching strategies
- Cache middleware and invalidation
- Memory management
- Query optimization

Professional Development

- API documentation with Swagger
- Comprehensive testing suite
- Health monitoring system
- Error tracking and logging

Production Deployment

- Docker containerization
- Environment configuration
- Load balancing with Nginx
- Database clustering
- SSL/TLS security

Monitoring & Maintenance

- Health checks for microservices
- Performance metrics
- Logging strategies
- Cache statistics

Congratulations!

You've completed the **complete Node.js backend mastery course**! You now have:

- ✅ **Solid Foundation** (Chapter 1) - Express, MongoDB, Authentication
- ✅ **Advanced Features** (Chapter 2) - File uploads, Email, Relationships
- ✅ **Production Ready** (Chapter 3) - Real-time, Caching, Testing, Deployment

Your Backend Can Now Handle:

- Thousands of concurrent users
- Real-time communication
- File uploads and processing
- Advanced caching strategies
- Comprehensive testing
- Production deployment
- Monitoring and health checks

What You've Built:

A **scalable, secure, production-ready backend** that could power:

- Social media platforms
- E-commerce applications
- Chat applications
- Content management systems
- API-first applications

Next Steps:

- Deploy to cloud platforms (AWS, Google Cloud, Azure)

- Implement microservices architecture
- Add GraphQL API layer
- Integrate AI/ML features
- Build mobile app backends

You're now a Node.js backend expert! 🎉🚀

Ready to build amazing applications!

1 2 3 4 5 6 7